

Design Patterns (Day -5)



Authorized & published by
Itlize Global LLC

Design Patterns

Design patterns are reusable solutions to common problems that arise in software design. They are templates for solving design problems that have proven to be effective in various situations

- Provide proven solutions to common design problems
- Promote a common vocabulary and shared understanding among developers
- Save time and effort in creating solutions
- Promote a modular and maintainable code base
- Are applicable to a wide range of design problems, regardless of platform or language.

Design patterns are not specific to any programming language or technology.

Factory Method Design Pattern

The factory method design pattern abstract the process of object creation and allows the object to be created at run-time when it is required. Factory method is a Creational Pattern . It is used to create objects.

In Factory pattern, object can be created without exposing the creation logic. In this pattern, an interface is used for creating an object, but let subclass decide which class to instantiate. The creation of object is done when it is required. The Factory method allows a class later instantiation to subclasses.

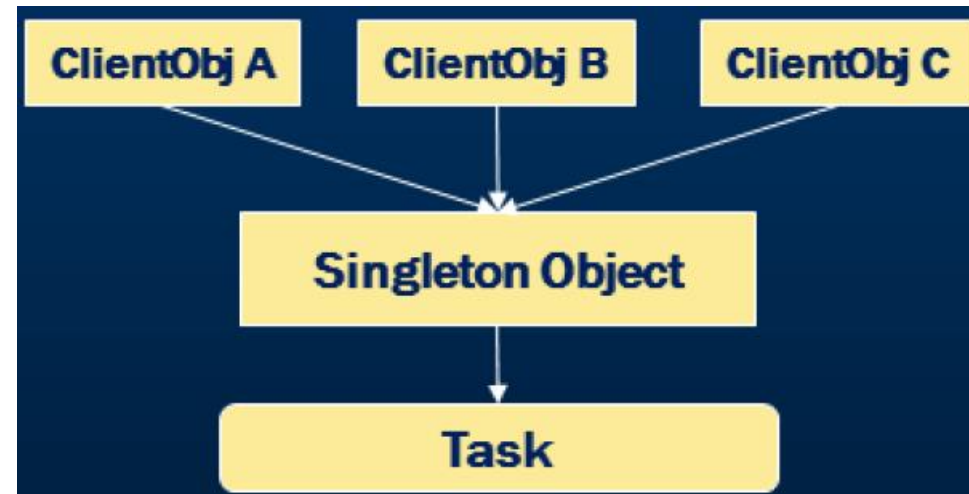
In short, factory method design pattern abstract the process of object creation and allows the object to be created at run-time when it is required.

Benefits

- Single Responsibility Principle
You can move the product creation code into one place in the program, making the code easier to support.
- Prevent tight coupling between the creator and the concrete products.
- Open/Closed Principle
You can introduce new types of products into the program without breaking existing client code.

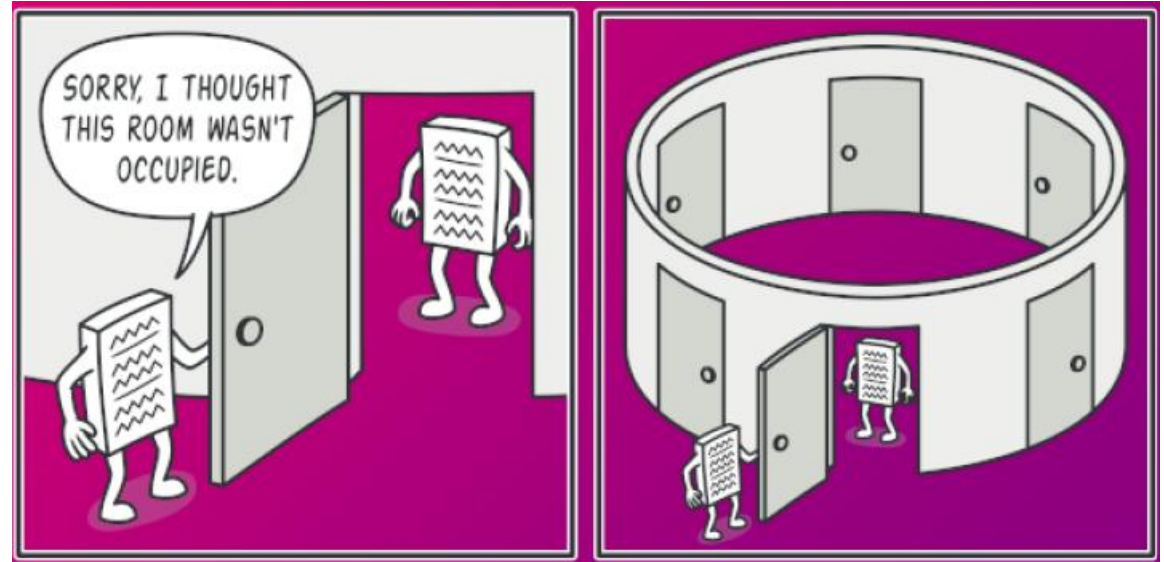
Singleton Design Pattern

Singleton is a design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.



Why Singleton

- To Ensure A Class Has Just A Single Instance
- To Provide Global Access Point to That Instance



WHEN: A class in your program should have just a single instance available to all clients.

How to Implement Singleton Design Pattern

- Declare a private constructor for the class for which you want to make Singleton to prevent other objects from using the new operator with the Singleton class.
- Declare a static variable for that class.
- Create a static method for that class that acts as a constructor and assign the object to the declared static variable.

```
namespace ConsoleApp1
{
    public class Volkswagen
    {
        private Volkswagen()
        {
        }

        public static Volkswagen obj;
        public static Volkswagen getInstance()
        {
            if (obj == null)
            {
                obj = new Volkswagen();
            }
            return obj;
        }

        public string getDetails()
        {
            return "Golf";
        }
    }
}
```

Dependency Injection Design Pattern

- Dependency Injection (DI) is a software design pattern that allows us to develop loosely coupled code.

BENEFITS OF USING DEPENDENCY INJECTION

- Maintainability
- Readability
- Testability
- Flexibility

